

Elevated Research Plan

Risk-Controlled Multi-Agent Negotiation for Dynamic GPU Scheduling

A detailed methodology with explicit models, formulas, baselines, and validation procedures

Research focus: Alibaba V2020 trace-driven evaluation of market-gated demand-supply agent negotiation

Executive Summary

This research studies whether role-separated AI-agent negotiation can improve the reliability of dynamic GPU allocation compared with a centralized single-LLM scheduler.

The proposed system separates job-side marginal utility from cluster-side opportunity cost. A demand agent evaluates urgency and the expected benefit of additional GPUs, a supply agent evaluates scarcity and cluster-wide cost, and a referee reconciles the two positions after one structured critique round.

A lightweight multi-unit auction handles routine states. Full negotiation is invoked only when deadline risk, uncertainty, conflict, starvation, or infeasibility exceeds a threshold. A quality-aware cache reuses previously validated decisions, while a non-repairing deterministic validator and fallback scheduler preserve execution safety.

The design explicitly addresses the major risks of this research: missing counterfactual ground truth in Alibaba V2020, unequal inference budgets, correlated agent errors, arbitrary credits, unrealistic utilization metrics, excessive resizing, stale cache reuse, and utilization-SLA trade-offs.

1. Refined Research Objective

The primary objective is to determine whether structured demand-supply negotiation reduces allocation regret and SLA violations relative to a single LLM, while maintaining or improving useful GPU utilization under equal information and inference budgets.

Core research question: Under equal information and inference budgets, can demand-supply negotiation reduce allocation regret and SLA violations compared with a centralized single-LLM allocator while maintaining or improving useful GPU utilization?

The contribution should be framed as a controlled decomposition of two conflicting quantities: job-side marginal utility and cluster-side opportunity cost. It should not be framed merely as using more agents or allowing agents to talk.

2. System Model

Let the cluster contain G allocatable GPU units. A unit can represent one physical GPU, a fixed fractional-GPU quantum, or one compatible MIG instance. At scheduling epoch t , the active job set is J_t . The scheduler produces the allocation for the next epoch while the current allocation is executing.

$$A_{t+1} = \{ g_{j,t+1} \mid j \in J_t \}$$

$$\sum_j g_{j,t+1} \leq G$$

$$g_j^{\min} \leq g_{j,t+1} \leq g_j^{\max}$$

For MIG experiments, capacity is represented separately for each profile k :

$$\sum_j x_{j,k,t} \leq C_{k,t}$$

The scheduler operates in three-minute epochs. During epoch t , allocation A_t is executed while the scheduler computes A_{t+1} . Future arrivals remain hidden except in a separately labelled oracle experiment.

3. Counterfactual Job-Progress Model

3.1 Main methodological risk

Alibaba V2020 records behavior under the allocation that actually occurred. It does not directly reveal how every job would have performed under alternative GPU allocations. Therefore, raw trace utilization cannot be treated as counterfactual ground truth for marginal GPU benefit.

3.2 Primary mitigation: calibrated saturation model

For each job j , define its progress rate under g GPUs as a saturating function:

$$p_j(g) = p_j^{\max} (1 - \exp(-\kappa_j g))$$

Here, $p_j^{\max}$ is the maximum achievable progress rate and κ_j controls scaling efficiency. Jobs with larger κ_j benefit more rapidly from additional GPUs, while jobs with smaller κ_j saturate earlier.

$$\Delta p_j(g) = p_j(g+1) - p_j(g)$$

The simulator uses this marginal benefit as counterfactual ground truth. The scaling parameter should be estimated from observed worker count, requested GPUs, GPU utilization, GPU-memory usage, workload class, and communication proxies. When direct estimation is unreliable, jobs should be assigned to poor, medium, and good scaling categories.

3.3 Robustness check

Repeat the main experiments under an alternative Amdahl-style scaling model:

$$p_j(g) = p_j(1) / (s_j + (1 - s_j)/g)$$

A result should be treated as robust only when its qualitative conclusion is consistent across both models and across sensitivity ranges for their parameters.

4. GPU Utilization Metrics

4.1 Risk: allocation is not useful work

A scheduler may achieve high GPU occupancy by allocating GPUs that contribute little progress. Therefore, allocated-GPU occupancy must be separated from useful GPU utilization.

4.2 Useful utilization

$$U_{\text{useful}} = [\sum_t \sum_j p_j(g_{j,t}) \Delta t] / [G T p_{\text{unit}}]$$

This metric represents normalized useful progress produced by the cluster. Also report physical GPU occupancy:

$$U_{\text{alloc}} = [\sum_t \sum_j g_{j,t} \Delta t] / [G T]$$

$$U_{\text{waste}} = U_{\text{alloc}} - U_{\text{useful}}$$

Reporting all three metrics prevents the paper from equating resource reservation with productive computation.

5. SLA Model

For each job j , define an isolated-runtime estimate T_j^{iso} and an SLA deadline based on an allowed slowdown factor λ :

$$d_j = r_j + \lambda T_j^{\text{iso}}$$

Evaluate multiple SLA tightness levels, such as λ in $\{1.2, 1.5, 2.0\}$.

$$v_j = 1[C_j > d_j]$$

$$\text{SVR} = (1 / |J|) \sum_j v_j$$

Also report normalized lateness so that minor and severe violations are distinguished:

$$L_j = \max(0, (C_j - d_j) / (d_j - r_j))$$

6. Routine Allocation Through a GPU Auction

6.1 Job-side marginal bid

Each job submits a separate value for each additional GPU unit. The bid combines expected progress improvement, SLA risk reduction, waiting-time fairness, and resizing cost:

$$b_{(j,k,t)} = \alpha \Delta p_{\text{hat}}(j,k,t) + \beta R_{\text{SLA}}(j,t) + \gamma W_{\text{wait}}(j,t) - \delta C_{\text{resize}}(j,k,t)$$

All components should be normalized to $[0,1]$. Marginal bids should be non-increasing so that later GPU units are not valued more than earlier units without explicit evidence.

$$b_{(j,1,t)} \geq b_{(j,2,t)} \geq \dots$$

6.2 Supply-side asking price

The supply price incorporates cluster scarcity, fragmentation, reserve pressure, and predicted arrival pressure:

$$a_{(k,t)} = \eta_1 \text{Scarcity}_t + \eta_2 \text{Fragmentation}_{(k,t)} + \eta_3 \text{Reserve}_t + \eta_4 \text{ArrivalPressure}_t$$

6.3 Market clearing

Sort job bids from highest to lowest and supply prices from lowest to highest. Allocate the largest number of units for which demand value remains at least as large as supply cost:

$$Q_t = \max \{ q : b_{((q),t)} \geq a_{((q),t)} \}$$

This mechanism adapts market-based resource allocation to discrete GPU or MIG units. It should be described as an adaptation rather than a direct implementation of proportional CPU-share markets.

7. Virtual-Credit Design

7.1 Risk: arbitrary budgets

If the initial credit or redistribution rule is arbitrary, auction outcomes may be driven by hand-tuned parameters rather than job value.

7.2 Mitigation: equal endowment and redistribution

$$B_{-}(j,0) = B_{-}0$$

$$B_{-}(j,t+1) = \min(B_{-}\text{max}, B_{-}(j,t) - P_{-}(j,t) + D_{-}(j,t))$$

$$D_{-}(j,t) = [\text{sum}_i P_{-}(i,t)] / |J_t|$$

Equal endowment limits prior bias, and periodic redistribution prevents jobs from permanently losing their ability to compete. Evaluate multiple initial budgets and redistribution policies to confirm that conclusions do not depend on one credit value.

8. Negotiation Trigger

8.1 Risk: unnecessary LLM calls

Invoking all agents every epoch increases cost and can make comparisons unfair. The system should negotiate only when the routine market path is uncertain or risky.

8.2 Seriousness score

$$\text{Gamma}_t = w_1 R_{\text{SLA}_t} + w_2 D_{\text{ambiguity}_t} + w_3 U_{\text{uncertainty}_t} + w_4 F_{\text{starvation}_t} + w_5 C_{\text{churn}_t}$$

All terms are normalized to [0,1]. Negotiation is invoked when Gamma_t exceeds threshold θ or when a hard trigger occurs.

- Predicted SLA violation
- No feasible auction allocation
- GPU or node failure
- Minimum requirements exceeding capacity
- Excessive starvation
- Validator rejection of a cached decision

A useful ambiguity measure gives high values when winning bids are close to supply prices, because those decisions are sensitive to prediction error:

$$D_{\text{ambiguity}_t} = (1/M) \sum_q \exp(-|b_{-}((q),t) - a_{-}((q),t)| / \tau_{\text{amb}})$$

9. Multi-Agent Negotiation Protocol

9.1 Demand agent

The demand agent evaluates job urgency, remaining work, scaling benefit, utilization history, and resizing cost. It outputs preferred allocation, marginal utility, SLA risk, evidence, and confidence.

9.2 Supply agent

The supply agent evaluates available capacity, scarcity, fragmentation, reserve requirements, queue pressure, and placement constraints. It outputs available surplus, reserve capacity, supply prices, protected jobs, evidence, and confidence.

9.3 One-round mutual critique

Each agent receives the other proposal and must identify unsupported assumptions, ignored constraints, numerical inconsistencies, missing evidence, and exaggerated confidence. One critique round is the default to control latency and avoid conversational drift.

$$D'_t = \text{Revise}(D_t, \text{Critique}_S(D_t))$$

$$S'_t = \text{Revise}(S_t, \text{Critique}_D(S_t))$$

9.4 Referee scoring rule

The referee compares candidate allocations using a fixed, auditable score rather than free-form preference:

$$J(A) = \lambda_1 U_{\text{hat_useful}}(A) - \lambda_2 \text{SVR}_{\text{hat}}(A) - \lambda_3 C_{\text{resize}}(A) - \lambda_4 F_{\text{unfair}}(A) - \lambda_5 C_{\text{invalid}}(A)$$

The referee outputs the exact next allocation and reports each score component, confidence, and the evidence used to resolve disagreement.

10. Reducing Correlated-Agent Errors

10.1 Risk

Agents using the same underlying model can share the same false assumption. Multi-agent discussion does not create true independence by itself.

10.2 Evidence-separated views

- Demand view: job progress, SLA slack, speedup predictions, utilization, and remaining work.
- Supply view: total capacity, fragmentation, reserve requirements, queue pressure, and topology.
- Referee view: both proposals, critiques, hard constraints, and a compact shared factual summary.

Each agent should provide its strongest evidence, one reason its own proposal may be wrong, and the condition under which it would change its recommendation. This converts confidence into an evaluable research variable rather than a rhetorical statement.

10.3 Diversity ablation

- Same model, different prompts
- Same model, different random seeds
- Heterogeneous models when available

These comparisons distinguish gains from role separation, stochastic diversity, and model diversity.

11. Deterministic Safety Layer

11.1 Non-repairing validator

$$V(A, S_t) = \text{product over } c \text{ in } C \text{ of } 1[c(A, S_t) \text{ is true}]$$

The constraint set includes total capacity, minimum and maximum allocations, integer or MIG-profile compatibility, GPU memory, node placement, topology, admission status, and resizing limits. The main experiment rejects invalid LLM outputs instead of silently repairing them.

11.2 What validation does and does not guarantee

- Prevents infeasible allocations, malformed outputs, and capacity violations.
- Does not guarantee fairness, high utilization, low future SLA risk, or good economic choices.

11.3 Deterministic fallback

When negotiation times out or fails validation, allocate minimum resources first and greedily assign remaining units by marginal progress, SLA urgency, and waiting time, minus resizing cost:

$$\text{Score}_j = \omega_1 \Delta p_{\text{hat}_j} + \omega_2 R_{\text{SLA}_j} + \omega_3 W_{\text{wait}_j} - \omega_4 C_{\text{resize}_j}$$

Fallback frequency is reported as a reliability outcome, not hidden inside the proposed method.

12. Cache Reuse with Safety Controls

12.1 State representation

$$z_t = [\text{load}, \text{active_jobs}, \text{urgent_ratio}, \text{mean_util}, \text{util_std}, \text{mean_slack}, \text{fragmentation}, \text{churn}]$$

12.2 Similarity and quality-aware retrieval

$$\text{sim}(z_t, z_i) = (z_t^T z_i) / (\|z_t\|_2 \|z_i\|_2)$$

$$Q_i = \alpha U_{\text{useful}_i} - \beta \text{SVR}_i - \gamma C_{\text{resize}_i} - \delta R_{\text{invalid}_i}$$

$$R_i = \text{sim}(z_t, z_i) Q_i \exp(-\lambda \text{age}_i)$$

Reuse only when the best retrieval score exceeds a cache threshold. Cache entries should be stored only after successful validation and execution.

12.3 Safe adaptation

- Map cached job categories to current jobs rather than matching raw identifiers.
- Preserve current minimum allocations.
- Adapt only surplus GPU assignments.
- Limit allocation change from the current epoch.
- Run the deterministic validator before execution.

Measure false reuse, defined as the fraction of cache reuses whose regret exceeds a chosen threshold. This is more informative than cache-hit rate alone.

13. Fair Comparison Against a Single LLM

13.1 Main risk

The multi-agent system may appear better simply because it receives more calls, tokens, or wall-clock time.

13.2 Equal-budget protocol

$$C_LLM = \alpha N_in + \beta N_out + \gamma T_wall$$

For each multi-agent configuration, give the single-LLM baseline approximately equal total input tokens, output tokens, samples, and wall-clock budget. Compare:

- Single LLM, one pass
- Structured single LLM
- Single LLM with self-critique
- Multiple independent single-LLM samples
- Demand and supply without critique
- Demand and supply with referee
- Full negotiation with critique

14. Allocation Regret

Because the simulator knows the progress model, it can compute an oracle allocation for each epoch:

$$A^*_t = \arg \max_{A \in F_t} J(A)$$

$$\text{Regret}_t = J(A^*_t) - J(A_t)$$

$$\text{NRegret}_t = [J(A^*_t) - J(A_t)] / [|J(A^*_t)| + \epsilon]$$

Normalized regret directly measures numerical decision quality and should be a primary outcome alongside SLA and utilization.

15. Reallocation Overhead

Frequent resizing can create unrealistic gains. Model reallocation delay explicitly:

$$C_resize(j,t) = c_0 + c_1 |g(j,t+1) - g(j,t)| + c_2 I_cross-node + c_3 I_MIG-reconfig$$

$$\Delta W(j,t+1) = p_j(g(j,t+1)) \max(0, \Delta t - C_resize(j,t))$$

Also enforce a resizing cooldown and evaluate one-, two-, and five-epoch cooldowns.

16. Starvation Protection

$$W_norm(j,t) = \min(1, (t - r_j)/W_max)$$

$$b_fair(j,k,t) = b(j,k,t) + \phi W_norm(j,t)$$

A job becomes protected when normalized waiting time exceeds a threshold. Report starvation rate and maximum waiting time in addition to Jain fairness.

17. Experimental Matrix

17.1 Main methods

- Fixed requested allocation
- Utilization-threshold reclaim
- Marginal-utility heuristic
- Auction-only
- Single LLM
- Single LLM with self-critique
- Multi-agent without critique
- Full negotiation
- Full negotiation with cache
- Oracle upper bound

17.2 Workload and sensitivity dimensions

Dimension	Recommended values
Offered load rho	0.3, 0.5, 0.7, 0.9, 1.1, 1.3
Prediction noise sigma	0, 0.1, 0.2, 0.4
Resize overhead	zero, low, medium, high
Cache threshold	0.70, 0.80, 0.90, 0.95
Trigger threshold	0.10, 0.20, 0.30, 0.40, 0.50
SLA slowdown lambda	1.2, 1.5, 2.0
Resource granularity	full GPU, abstract fractional unit, MIG profile

18. Statistical Analysis

Use paired trace windows and random seeds. Every scheduler should run on the same workload realization. Report mean, median, standard deviation, 95% confidence intervals, and effect sizes.

$$D_i = X_i - Y_i$$

$$\text{mean paired improvement} = (1/n) \sum_i D_i$$

Use a paired t-test when paired differences are approximately normal and the Wilcoxon signed-rank test otherwise. Apply Holm correction for multiple comparisons. Report Pareto dominance for utilization-SLA trade-offs rather than relying only on a weighted objective.

19. Strong Hypotheses

H1 - Role separation: Under equal inference budgets, demand-supply role separation reduces normalized allocation regret compared with a centralized single-LLM allocator.

H2 - Mutual critique: One round of bidirectional critique reduces unsupported claims and improves final allocation quality compared with independent demand and supply proposals.

- H3 - Safety:** Deterministic validation reduces executed infeasible allocations to zero, while fallback frequency reveals the residual unreliability of LLM decisions.
- H4 - Auction gating:** Auction gating reduces multi-agent invocation frequency and scheduling latency without a statistically significant increase in SLA violations.
- H5 - Cache reuse:** Quality-aware similarity caching reduces average and P95 scheduling latency while keeping SLA and regret within predefined non-inferiority margins.
- H6 - Saturation conditions:** Negotiation provides its largest benefit near cluster saturation, where marginal job utility and cluster opportunity cost are both significant.

20. Recommended Final Architecture

1. Observe Alibaba-based state and estimate counterfactual progress.
2. Apply admission control and protect minimum allocations.
3. Query the quality-aware cache.
4. If reuse is safe, adapt and validate the cached candidate.
5. Otherwise clear the multi-unit auction.
6. Compute seriousness and invoke negotiation only for critical states.
7. Run demand and supply agents in parallel.
8. Perform one critique round.
9. Use the referee to produce the exact next allocation.
10. Validate the candidate without repair.
11. Execute a valid candidate or use deterministic fallback.
12. Log progress, latency, regret, critique outcomes, and cache quality.

21. Risk-Control Summary

Risk	Specific mitigation
Missing counterfactual GPU ground truth	Calibrated progress model, alternative-model sensitivity analysis, and strict separation of observed and simulated variables.
Multi-agent gains caused by larger inference budget	Equal-token, equal-sample, and equal-latency comparisons.
Correlated agent mistakes	Role-specific evidence views, self-counterarguments, confidence logging, and diversity ablations.
Arbitrary auction credits	Equal endowment, redistribution, and credit sensitivity analysis.
Misleading utilization claims	Useful-progress utilization, allocation occupancy, and waste reported separately.
Excessive resizing	Explicit resize-cost model and cooldown constraints.

Unsafe LLM output	Non-repairing validator and fully logged deterministic fallback.
Stale cache reuse	Quality-weighted similarity, age decay, safe adaptation, and validation.
Utilization gains harming SLA	Pareto analysis and SLA non-inferiority tests.
Future trace leakage	Only currently observable information available to online schedulers.
MIG treated as continuous sharing	Discrete profile-specific capacity and compatibility constraints.
Negotiation latency	Selective triggering, parallel agents, one critique round, compact JSON, timeout, and cache reuse.

22. Publication Framing

The paper should present the contribution as a controlled decomposition of conflicting scheduling objectives, not as a claim that multiple agents are inherently more intelligent.

Recommended framing: Dynamic GPU scheduling requires estimating both the marginal benefit of additional resources to each job and the opportunity cost imposed on the cluster. We study whether explicitly separating these perspectives, allowing structured mutual critique, and reconciling them through a referee improves allocation reliability under equal inference budgets.

The decisive empirical test is whether perspective-separated negotiation reduces normalized allocation regret and SLA violations compared with a well-structured, equally funded single LLM, while maintaining or improving useful GPU utilization.